Prediction and Ranking of Companies

Ilyas Amaniss
Computer Science and Operations Research
Université de Montréal
Montréal, Qc
ilyas.amaniss@umontreal.ca

November 19, 2020

Abstract

The use of machine learning in the financial world is varied. At Caisse de dépôt et placement du Québec (CDPQ), an exploration of different prediction tasks was performed in order to determine a potential method for leveraging the financial data available in this institution. Using different models such as an Auto Regressive Integrated Moving Average with explanatory variables (ARIMAX), a gradient boosting and a Long short-term memory (LSTM), three tasks were implemented with the goal of accurately predicting the potential gains of investing within a company. Given a Universe of companies, the simulation performed showed that investing in the companies suggested by the gradient boosting model constructed yield to better 12 months returns on average compared to investing in the entire Universe of companies.

Contents

1 Introduction **3**
- 1.1 Machine Learning in Financial Institutions . . . 3
- 1.2 Caisse de Dépôt et Placement du Québec (CDPQ) . . . 3
- 1.3 Task Description . . . 3
- 1.4 Tools and resources . . . 4
- 1.5 Covid-19 pandemic . . . 4

2 Data Analysis **4**
- 2.1 Features and Description . . . 4
- 2.2 Data Engineering . . . 7

3 Literature Review **8**
- 3.1 Machine Learning in Finances . . . 8
- 3.2 Algorithms . . . 9
  - 3.2.1 ARIMAX . . . 9
  - 3.2.2 Gradient Boosting . . . 10
  - 3.2.3 Long Short-Term Memory (LSTM) . . . 11

4 Experiments **13**
- 4.1 Regression Task . . . 13
  - 4.1.1 Baseline . . . 13
  - 4.1.2 ARIMAX . . . 13
  - 4.1.3 Gradient Boosting . . . 17
  - 4.1.4 Gradient Boosting with Time Series . . . 19
  - 4.1.5 LSTM . . . 22
  - 4.1.6 LSTM with Clustering . . . 26
- 4.2 Classification Task . . . 28
  - 4.2.1 Baseline . . . 28
  - 4.2.2 Gradient Boosting . . . 29
  - 4.2.3 Gradient Boosting with Time Series . . . 32
  - 4.2.4 LSTM . . . 34
  - 4.2.5 LSTM with Clustering . . . 37
- 4.3 Ranking Task . . . 40
  - 4.3.1 Gradient Boosting . . . 40

5 Recommendations **44**

6 Conclusion **45**

1 Introduction

1.1 Machine Learning in Financial Institutions

As advancement is made in artificial intelligence and more specifically in machine learning, the areas where those technologies are applied also widen. One particular challenging field where it always seemed interesting to use machine learning, is in the financial markets. Indeed, the idea of automating the investment process, being able to build a model able to beat human investors and automatically making monetary gains probably came to mind at least once to most data scientists and investors.

The application of those data science methods can vary. Indeed, one could simply explore, compute and analyse some metrics specific to a company or a sector of investment in order to determine the long term investment potential of them. On the other side, some could use machine learning and the pros of fast computing to automate day trading and trade the stock market or the foreign exchange market.

Today, it is not an exaggeration to say that most important investment firms are currently implementing some AI's approaches to enlighten their investment decisions or are at least starting to head into this direction. The Caisse de Dépôt et Placement du Québec (CDPQ) is no exception to this trend.

1.2 Caisse de Dépôt et Placement du Québec (CDPQ)

Caisse de dépôt et placement du Québec (CDPQ) was founded in 1965 and is an institutional investor. This pension fund manages several public pension plans, insurance programs in Quebec, but also has investors worldwide. The Caisse's mandate is to invest the assets of Quebecers to ensure their retirement, while contributing to the development of Quebec's economy. With more than $333 billion in assets under management, of which around $150 billion is invested in the stock markets, the Caisse is Quebec's largest investor.

1.3 Task Description

For this master's project, CDPQ was interested to know if, with the provided data of a particular sector of investment including different companies, the best (or worst) companies could be highlighted with the goal of making smarter investment decisions. Moreover, since this project's clients are mainly investors which do not necessarily have a deep background in artificial intelligence, it was mandatory to also be able to understand the behavior and the decision making process of the models built. Thus, the trade-off between a model's complexity and its performance always had to be considered throughout the development and research of this project.

Also, unfortunately, for privacy reasons and since this project is an ongoing initiative, the sectors, companies or any other data or information used in this project will be kept private.

1.4 Tools and resources

CDPQ had plenty of tools and resources available for this project. Indeed, I was provided with an account to access the Amazon Web Services (AWS). Within those resources, I mainly used the cloud machine-learning platform Amazon SageMaker and Amazon S3. SageMaker is a service that provides a developer and data scientist with the ability to build, train, and deploy machine learning models. So, SageMaker's notebooks and Jupyter Lab were used to monitor the training of the models, visualize the graphs made and the results of the models' tests. On the other hand, Amazon S3, also named Amazon Simple Storage, is a service offered by AWS that provides object storage through a web service interface. So, S3 was used to access and read CDPQ's data.

1.5 Covid-19 pandemic

The internship, starting at the very beginning of the Covid-19's pandemic and isolation in May 2020, had to be done remotely. Indeed, when I arrived at CDPQ, many measures were already implemented in order to keep everyone safe, but also to allow everybody within the company to continue working in the best possible conditions. After receiving my working station by mail, I was presented with the project, the data and the tools in disposition. All meetings were virtual and conducted using Microsoft Teams. I was able, whenever needed, to contact my coworkers or even my team's leader if required. Also, the meetings with my Mila's supervisor were also done remotely. Indeed, a bi-weekly meeting was scheduled with my Mila's supervisor and a coworker within CDPQ to share ideas, give updates about the project and to receive feedback.

2 Data Analysis

The data provided by the Caisse, called a Universe, contains multiple features of more than 2000 companies within the same sector of operation. Each of the companies then belongs to a sub-sector of this specific sector of interest.

2.1 Features and Description

Totaling 148 298 daily entries from 1994 to 2020, multiple companies' data is listed, such as belonging to the SP400, SP500 and SP600, the 3 months, 6 months, 12 months, 24 months and 36 months returns of this company, the sub-sector which it belongs to and multiple other features which will be kept hidden. Specifically, the financial return of a company is how much money an investor gets back from investing in this particular company. This number, theoretically going from -100% to infinity, shows how much profit or loss you get back from investing in this company.

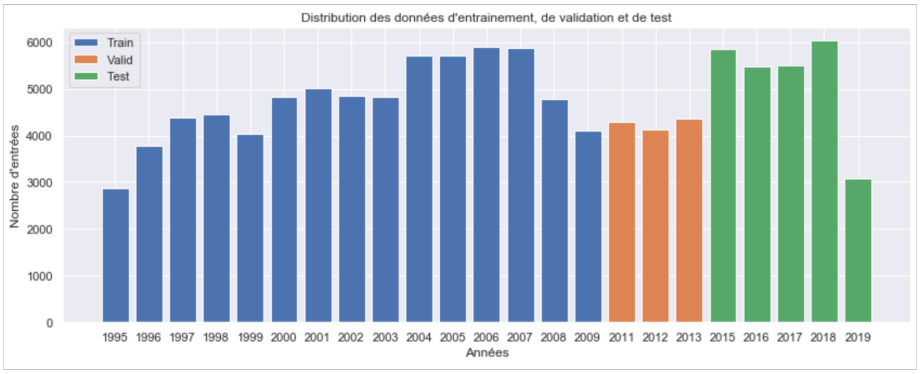


Figure 1: Number of entries per year

Also, it is important to note that, each day, a different number of companies could be listed. To be recognized, each company has a specific and unique ID which is partially based on its sub-sector of activity. Indeed, a company created, for instance, in May 6th 1998 will not appear before this date. Additionally, some companies could also disappear from the dataset after some time for different reasons such as considering this company not belonging anymore to this sector of activity or simply because the company declared bankruptcy and does not exist anymore.

Another particularity of this dataset is that it was formed using multiple data providers and merged together. Indeed, since the different datapoints come from different sources, this leaves some features, for some specific companies with *not-a-number* (NaN) values. For instance, the first source could have features from 1996 to 2020 for some companies and the second source features from 1994 to 2010. Thus, this leaves some companies in years 1994, 1995 and 1996 without values for those specific features. Mostly, those NaN values were left in the dataset and dealt with specifically in each experiment.

For this project, the target which mostly interested CDPQ was the 12 months return of a particular company. However, since the goal of this project was to find the companies with the best potential returns, the excess return was also used for some experiments. Note that the excess return is the difference between a company's return and the average return of all the companies within this Universe.

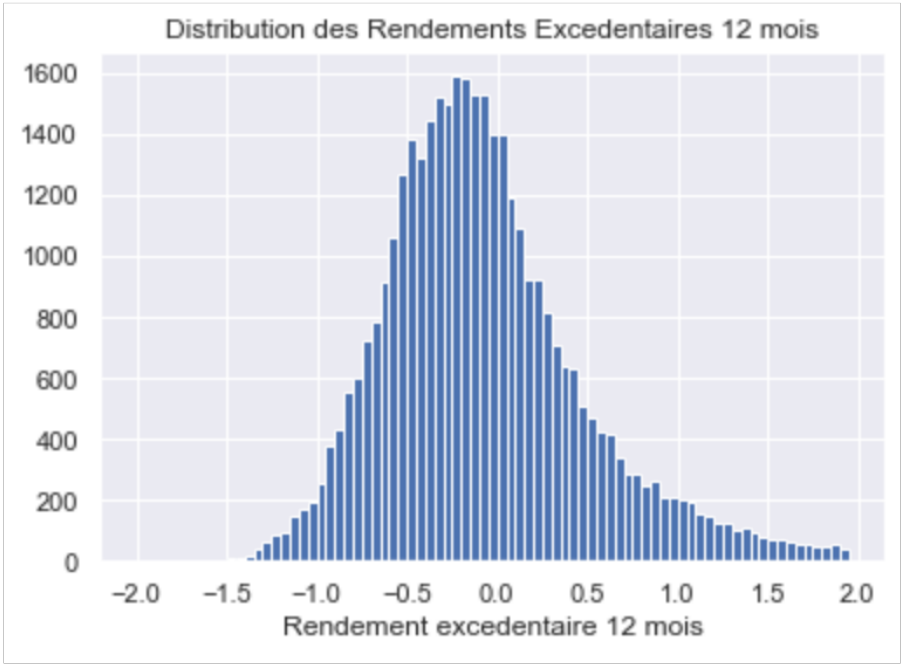


Figure 2: Distribution of company's 12 months Excess Return

As shown in figure 2, the 12 months excess returns' distribution of the companies resemble a Gaussian distribution. According to this figure, most companies' 12 months excess return are between -50% and 50% with some companies returning almost 200% more than the average of the companies and others performing well under the average: with returns close to -150%. A value of -150%, seems counter intuitive, however there is a distinction to be made between a return and an excess return. Indeed, let's for instance consider an investor with 100$ to invest. This person can either choose to separate this 100$ and equally invest in all the companies of this Universe or choose to put the entire 100$ in this particular company. Say, after 12 months, by investing equally in all the companies, a return of 100% is made: this investor now has 200$ in his pocket. On the other hand, this particular company had, after 12 months, a return of -50%, thus leaving this investor with 50$ left in his pocket. The difference of 150% between the two choices (the average of all the companies and the particular company) is what is called the excess return of this company.

The boxplot of figure 3, grouped the companies by year. It presents the median with a line and the box shows the lower and upper quartile in this particular year.

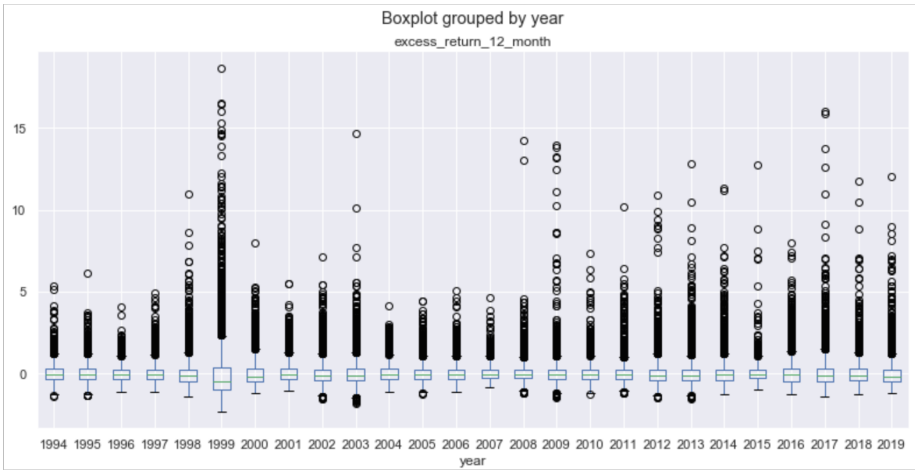


Figure 3: Yearly Distribution of company's 12 months Excess Return

This gives a visual representation of the differences between each year. Overall, it is easy to see that the median, usually close to zero, is approximately equal every year. However, many companies are getting unusually very good performance, such as in 1999 with some excess returns close to 200% shown by the black circles. These points could be considered outliers within this dataset and although those companies seem very interesting to invest into, they are usually very volatile and risky companies and not what interests CDPQ.

2.2 Data Engineering

Since the provided data is considered a time series, multiple particularities had to be considered. So, some pre-processing steps were applied to account for the 12 months target and the nature of the data.

First, the data was reduced and simplified in order to transform the daily values to monthly values. This was done since for almost everyday of the year (excluding weekends, since the markets are closed) data are provided for companies. However, because of some properties of time series and finances in general, the features of a company do not change much from one day to another. This is expected since, expect for very volatile stocks, a company's valuation or other features won't make big spikes in under 24 hours. So, in order to reduce the size of the dataset and reduce redundancy, monthly datapoints were taken instead of daily datapoints.

Additionally, because of the feature's and target's closeness still present from one month to another, it was important to correctly separate the data between the training, validation and testing sets. Indeed, with the goal of reducing any look-ahead bias and leakage of data between the training, validation and test sets, purging had to be applied during this project. In order to reduce leakage of information between the sets, all data which overlapped in time were removed [1].

Indeed, because monthly data points are still closely related and the target was the 12 months return, a one year purge had to be applied between the training and validation sets, but also between the validation and test sets.

Moreover, because it is a time series, it is arguable that today's return could be correlated to the future 12 month's excess return. So, a technique well-known in time series forecasting was applied, namely the recurrent sliding windows. Mainly with the recurrent sliding window method, the target of interest y_t is fed back as input to make the prediction for y_{t+1} [2]. So, in addition to the features $x_{0:n}$, the current y_t was also given as input. In other words, today's 12 months return of the company was also given as a feature to the models in order to predict y_{t+12} (the 12 months return).

So, throughout this project, the training set used included the years 1994 to 2009, the validation set included the years 2011 to 2013 and the test sets, the years from 2015 to 2019 as shown in figure 1. As explained previously, the years 2010 and 2014 were purged from the data to reduce any look-ahead bias.

It is important to note that some techniques exist in order to make use of the totality of the dataset, such as the expanding window and the sliding window methodologies [3]. The idea is to change the training, validation and test sets during training, while keeping the gap between the sets. The expanding window technique was actually experimentally tested at the very beggining of the project but not kept as the final training methodology since it provided too much overhead for the results gained. So, a classic training, validation and test sets methodology was kept in order to fine-tune the search of hyper parameters.

3 Literature Review

3.1 Machine Learning in Finances

The use of machine learning in financial institutions or simply in finances is varied. Indeed, some use artificial intelligence in order to directly predict stock prices in order to determine if this company is interesting to invest or, on the other hand, if a stock currently in one's portfolio should be sold. Usually accompanied by simulation with historical data in order to determine the estimated returns of the models, multiple scientists proved that neural networks, for instance, had a lot of potential in giving good returns by following their predictions [4]. Sometimes, such models are also feed, not only the company's historical values or data, but also events and even texts published in the media in order to have more accurate predictions [5].

Indeed, by using a long short-term memory (LSTM) for a stock prediction task, some interesting results were achieved: the model was able, with an accuracy of 55.9% to determine if a particular stock was going up or not in the near future [6]. This slight edge of 55.9% could allow, with the right consideration of transaction prices and fees, to have good returns when using this model. Another model, the Auto Regressive Integrated Moving Average (ARIMA), also proved to be able to compete with current stock prediction technique and had a strong

potential for short-term prediction [7].

3.2 Algorithms

This section will thus summarize not only the algorithms used, but also the modifications and variants tested of those popular machine learning models in order to answer the specific needs of this project. Mainly, three models were implemented during this project: the Auto Regressive Integrated Moving Average with explanatory variables (ARIMAX), the XGBoost, which is a framework implementing a decision-tree based gradient boosting algorithm and the Long short-term memory (LSTM) being a specific flavor of an artificial recurrent neural network (RNN).

3.2.1 ARIMAX

Let's first define what an autoregressive model (AR) and a moving average (MA) are.

An autoregressive model will model y_t as a combination of its past values. AR(p), where p is the number of parameters of the model, is commonly represented as:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + ... + \phi_p y_{t-p} + \epsilon_t$$

So, for instance, if one would want to use an autoregressive system to model the current stock price of a company, today's stock price would be a combination of its past's stock price plus an error term ϵ.

A moving average model on the other side will model y_t as being a combination of past error terms. Let, $\epsilon \overset{iid}{\sim} N(\mu, \sigma^2)$ meaning that the error terms are identically, independently distributed, each with a normal distribution having mean 0 and the same variance. Using a MA(q) model, where q is the number error terms, we can model y_t as:

$$y_t = \phi_0 + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \theta_3 \epsilon_{t-3} + ... + \theta_q \epsilon_{t-q}$$

So, by combining the AR and MA models, the ARMA (p,q) can be used and takes the form:

$$\begin{aligned} y_t = {} & \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + ... + \phi_p y_{t-p} + \epsilon_t \\ & + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \theta_3 \epsilon_{t-3} + ... + \theta_q \epsilon_{t-q} \end{aligned}$$

Furthermore, the ARMA(p,q) can be extended to a ARMAX(p,q) which also includes explanatory variables (which are the features available in this dataset) with the goal of adding the linear effect of those variables to the studied time series. For instance, in addition to use the past stock price and errors, some features such as the inflation evolution for the past years or the change in unemployment rate can be added as inputs with the hope of having an even

better representation of the studied time series. So, with the new features $x_1, x_2, ..., x_r$ and using the ARMAX(p,q), y_t can now be modelled as:

$$y_t = \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{k=1}^{r} B_k x_{tk} + \epsilon_t + \sum_{j=1}^{q} \theta_j \epsilon_{t-j}$$

In lag operator notation [8], its condensed form can be written as:

$$\phi(L) y_t = c + x_t' B + \theta(L) \epsilon_t$$

Also, in order to use the ARMA or ARMAX models, the assumption that the time series is stationnary is also taken. In other words, the response series is stable if the roots of the homogeneous characteristic equation of $\phi(L) = L^p - \phi_1 L^{p-1} - \phi_2 L^{p-2} - ... - \phi_p L^p = 0$ lie outside of the unit circle [9]. However, if the response series y_t is not stable, one can differentiate it to form a stationary ARIMA and also ARIMAX model. So, the ARIMAX(p, d, q) model thus takes the form:

$$\phi(L) y_t = c^* + x_t' B + \theta^*(L) \epsilon_t$$

Where, $c^* = \frac{c}{(1-L)^d}$, $\theta^*(L) = \frac{\theta(L)}{(1-L)^d}$ and d is the degree of integration.

3.2.2 Gradient Boosting

Gradient boosting revolves around the use of multiple simple functions called base learners and fitting them to the task in multiple iterations [10]. In other words, it is an ensemble method combining multiple weak prediction models.

Let's consider a classic mapping problem where the training samples of known values $(y_i, x_i)_1^N$ and where the goal is to find $F*(x)$ mapping x to y, where over the joint distribution of all (y, x) values, the expected value of a loss function $\phi(y, F(x))$ is minimized. Thus having:

$$F^*(x) = \arg\min_{F(x)} E_{y,x} \phi(y, F(x)) \tag{1}$$

By defining the functions $h(x, a)$ which are the weak learners with input x and parameters $a = (a_1, a_2, a_3, ...)$, boosting approximates $F*(x)$ by an additive expansion [10]:

$$F(x) = \sum_{m=0}^{M} B_m h(x, a_m) \tag{2}$$

Where B_m^M (the expansion coefficients) and a_m^M the parameters are fitted to the training data. Indeed, we first start with an initial $F_0(s)$ and for $m = 1, 2, ..., M$ we have [10]:

$$(B_m, a_m) = \arg\min_{B,a} \sum_{i-1}^{N} \phi(y_i, F_{m-1}(x_i) + B h(x_i, a)) \tag{3}$$

and

$$F_m(x) = F_{m-1}(x) + B_m h(x, a_m) \tag{4}$$

In order to find B_m and a_m, the gradient boosting (Friendman, 1999) can be used :

$$a_m = \arg\min_{a,p} \sum_{i=1}^{N} (y_{im} - ph(x_i, a))^2 \tag{5}$$

where

$$y_{im} = -[\frac{\partial \phi(y_i, F(x_i))}{\partial F(x_i)}]_{F(x)=F_{m-1}(x)} \tag{6}$$

is the current pseudo-residuals. So, using $h(x, a_m)$, B_m can be found with:

$$B_m = \arg\min_{B} \sum_{i=1}^{N} \phi(y_i, F_{m-1}(x_i) + Bh(x_i, a_m)) \tag{7}$$

Thus, instead of a solving a difficult optimization problem (3), we can solve a problem based on least square (5) and on a single parameter optimization (7) which is based on ϕ (the general loss criterion [10].

The effect of using multiple weak learners is to obtain a model which has a high bias, but low variance. Also, by setting the weak learners as regression trees, one can form the Gradient Tree Boosting algorithm [10] and one of the frameworks implementing this algorithm is the XGBoost [11].

On the other hand, depending on the task at hand and the dataset, the other solution would be to aim to have a low bias, but high variance. This is where algorithms such as the Random Forest [12] come to use.

The XGBoost framework has multiple configurable hyper parameters such as the number of estimators, the maximum depth of the trees built during training, the minimum loss reduction required to make a further partition on a leaf node of the tree (which is named the gamma value) and the subsample ratio of the training instances which controls the percentage of random samples from the training data prior to growing trees. Those hyper parameters will be explored in each of the experiments involving this framework.

3.2.3 Long Short-Term Memory (LSTM)

An artificial neural network, is a 'computational model' able to learn, adapt, cluster, generalize or cluster data [13]. One example of a common neural network is recurrent neural networks or feedforward neural networks. Due to the internal structure of a traditional feedforward neural network, it lacks the ability to have persistent information throughout each input. To put it simply, a traditional feedforward neural network goes only forward and never feedback to itself. Thus,

without persistent information, a feedforward neural network would not be able to link previous information with the present information, an important feature to have in order to accurately predict future stock movements. This is where recurrent neural networks come to hand.

Recurrent Neural Networks address this problem by not assuming that all inputs and outputs are independent of each other [14]. A recurrent neural network node contains a feedback loop which allows it to loop back information every step. Due to the structure of a recurrent neural network, the information retained in the recurrent neural network will pass through multiple stages of multiplication. According to Bengio [14], since each time steps of a recurrent neural network is related to one another by multiplications, the derivatives used in backpropagation could potentially reach 0, hence why it is called the vanishing gradient problem.

To solve the vanishing gradient problem, Hochreiter & Schmidhuber introduced a new type of recurrent neural network called Long Short Term Memory networks (LSTM) [15]. Structure wise LSTM networks look fairly similar to traditional recurrent neural networks, however unlike the traditional recurrent neural networks, LSTMs have the capability to decide when to store information and when to discard information. According to them, the memory like feature and the revamp of the internal structure of the LSTM node, solved the vanishing gradient problem.

There are three steps inside a LSTM node and the formulas used within them are as follow:

$$
\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
h_t &= o_t \circ \sigma_h(c_t)
\end{aligned}
$$

where x_t is the input vector to the LSTM unit, f_t is the forget gate's activation vector, i_t is the update gate's activation vector, o_t is the output gate's activation vector, h_t is the hidden state vector, c_t is the cell state vector, W, U and b are the weight matrices and bias parameters [16].

The first stage is called the forget gate layer. The output of the previous step (h_{t-1}) and the input (x_t) will be inputted into a sigmoid function which will output a value between 0 and 1, where 0 means the node should forget the previous information (C_{t-1}) and 1 means the node should keep the previous information.

In the next step, the LSTM will determine if the input information should be stored in the memory. There are two parts to this step, first part is the tanh layer and the second part is the sigma layer. The tanh layer gets a value between -1 to 1 from the input signal. One can think of this as the response of a node getting excited from the input. The sigmoid portion also known as the "input

gate layer" will determine if the output of the tanh layer should be added to the memory.

Finally, in the last step, the sigmoid will determine how much of the memory information is the output of the node.

4 Experiments

This section will include the different experiments made during this project. To structure this project, those experiments were divided into three distinct sections: the regression task, the classification task and the ranking task. Indeed, different supervised tasks could be used to build models with the goal of making monetary gains in the financial markets. Thus, during this internship, three were implemented and explored using different models.

4.1 Regression Task

The first supervised task implemented was a regression task. The goal was thus to directly predict the future returns of a company. The insight behind this idea is that if a model could predict with perfect accuracy the gains or loss from investing in a particular company, it would be quite easy to leverage this model to make revenue, but also to stop one from investing in particular companies.

Five different implementations of models were tested for the regression task. Also, in order to fit the different input format of the models implemented in this experiment, some preprocessing steps were necessary. Indeed, first the dataset had to be separated in training, validation and test sets. Then, the features considered the most important by the investment team were selected for the models and the target was set to be either the 12 months returns of the company or the excess return, depending on the experiment. Finally, further preprocessing was applied depending on the model used and will be specifically explained in the following sections.

4.1.1 Baseline

The baseline taken for the regression task was based on the assumption that the 12 months returns would be close to the average of the past months. So, for each company, the average of the past 5 months was taken and compared using the mean squared error (MSE), with the 12 months return of the company. Using this model, for the complete universe, an average MSE of 0.6649 was obtained. This value will thus be compared with the other algorithms to have an idea of the precision of the models built.

4.1.2 ARIMAX

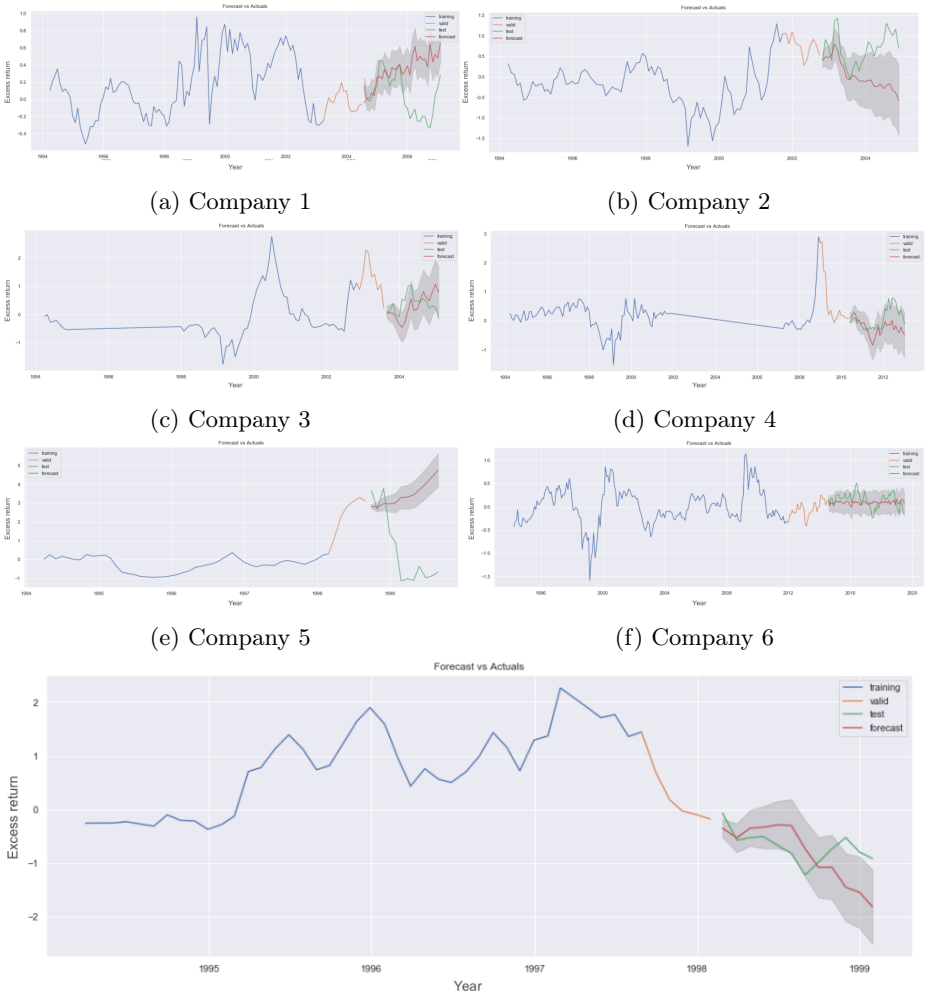
Preprocessing As explained in section 3.2.1, the ARIMAX takes as input a time series and multiple explanatory variables (features of this dataset). Since the companies are not necessarily correlated, the data was separated and one

time series per company was formed. Another solution would've been to take an average of the companies within the Universe and make a time series out of it. However, since the objective of the project was to determine if a specific company is worth investing into, this idea was rejected. Another challenge, as explained in section 2.1, is that a company could appear and disappear from the dataset as time goes by. Since the ARIMAX model requires a minimum of data in order to fit a proper function, all companies which didn't appear for at least 36 months within the 24 years of data were rejected and thus couldn't have an ARIMAX model fitted to their data.

Training Methodology Since the data is specific to each company, the training, validation and test sets presented in figure 1 couldn't be used. Indeed, there was no guaranty that a company would appear within those specific years. For instance, a company could be present from 1994 to 2009 (thus respecting the threshold of 36 months), but then be absent since then. So, instead, after forming a company's time series, 70% of its data was set as training, 10% as validation and 20% for test. A validation set was required for the ARIMAX model in order to find the best values of p, d and q explained in section 3.2.1. The values tested for each of those hyper parameters were either 0 or 1. This was necessary since the training process was done automatically and the number of data points from one company to another could change dramatically. So, a range which would accommodate most companies had to be used. Indeed, using a degree of differentiation or a number of autoregressive terms too high with a model having a very limited number of datapoints would throw a non-converging exception for instance. On the other hands, by increasing the minimum number of datapoints required in order to be considered for this model (currently set to 36), too many companies would be rejected. So, for each company's ARIMAX model, a grid search was applied in order to test the eight hyper parameter combinations. The mean squared error between the prediction and the actual value was taken as the metric to determine the models' performance.

Also, because of the sometimes limited number of training points and the model's hunger for training data in order to perform reasonably well, after finding the best set of hyper parameters, all the points from the validation set were also added as training. Thus, using the best hyper parameters found, the model was trained a second time with more data.

Results By requiring a minimum of 36 months of data from a company in order to fit an ARIMAX model, only 1021 companies out of the 2063 from the original dataset were considered. So, 1021 ARIMAX models, each with a hyper parameter search, were constructed and tested. Figure 4 shows 7 models, each from a different company, that were implemented. Many peculiarities are present withing those graphics.



(a) Company 1

(b) Company 2

(c) Company 3

(d) Company 4

(e) Company 5

(f) Company 6

(g) Company 7

Figure 4: Seven different ARIMAX models, each specific to a company

First, we can see, as explained previously, that the companies were present during different periods and for different duration. Indeed, as we can see in the figure above, some test values stopped around 2005 for some companies (companies 2 and 3) while others had values going almost until 2020 (company 6). Also, the behavior from those seven companies' returns are totally different. Moreover, each of those time series seem to behave in a random fashion: no clear behavior can be drawn from simply looking at those graphs, thus making the prediction somewhat challenging, at least for someone not involved in the financial markets.

ARIMAX	Hyper Parameters			Best Validation MSE	Test MSE
	d	p	q		
1	1	1	1	0.2913	0.3512
2	0	0	1	0.3996	0.1874
3	1	0	1	0.5179	0.1067
4	1	1	1	5.5164	3.1049
5	0	1	0	0.2388	0.0690
6	0	0	1	0.2389	0.0595
7	1	1	0	0.0687	0.0529

Table 1: ARIMAX Validation and Test MSE

For most of those models, we can see that the forecast, shown in red, is reasonably close to the test datapoints (in green). More specifically, we can see in table 1 above, by comparing the validation and test MSE, but also by visually looking at the predictions, that the models were reasonably able to model the company's time series. Furthermore, naturally, the closer we are from the training and validation data, the easier it is for those models to make accurate predictions due to the particularities present in time series data. For instance, we can see that the ARIMAX built for company 7, was able to forecast and follow the ups and downs of the company's real return and the overall downtrend of the company. However, we can also see that the further we are from the training data, the worst the predictions are getting. Indeed, for the companies 1 and 2, the initial forecast were closely related to the real values, however, after some times, the actual and forecast values diverged, showing one of the weakness of the ARIMAX models.

Analysis The ARIMAX, even though theoretically very interesting, also has many limitations. First, this model is not practical for the specific objective of this project. Indeed, when considering if a company is interesting to invest in or not, an ARIMAX model would need to be created and fitted to the company's data. However, in cases where not enough data exists for this company, no model can be created. It is true that the 1021 ARIMAX models built could be used to analyze those 1021 specific companies, however, based on the overall quite poor results obtained, especially for longtime forecasts which is what we are particularly interested in, such models wouldn't be trusted by a professional investor.

Also, since many models need to be created, it is a challenging task to save them and keep them organized. Furthermore, if one would want to use such models in production, once a month for instance, all the models would need to be updated with the updated data available at CDPQ. However, because of the multitude of models (one per company), retraining all of those models or even only the models specific to the companies which interests the investor would be a tedious task.

4.1.3 Gradient Boosting

The next model tested was the gradient boosting using the XGBoost framework. This model, fast to train, is very popular in the industry because of its usually good performance but also because of its easy interpretability: something important for the CDPQ's investors and thus this project. After training a model using the library Scikit-learn, one can access multiple modules of the trained XGBoost such as the different trees used in the model, the feature importance and many other specifications allowing the scientists to understand the decisions made by the model. This allows others having less knowledge in machine learning to understand the reasoning behind the model's predictions.

Preprocessing Here, contrary to the ARIMAX model, all the companies were feed to the same model. Using the current's month available features $[x^{(1)}, x^{(2)} \dots x^{(n)}]_{(t)}$ and $y_{(t)}$, the goal was to predict $y_{(t+12)}$, since the prediction is the 12 month's excess returns. So, all companies's monthly features and 12 months excess returns were feed to this single model.

Training Methodology As explained in section 2.2, the dataset was separated in a training, validation and test set: 15 years, 3 years and 5 years of data for each sets respectively. Using Scikit-learn's random search in order to find the best hyper parameters described in section 3.2.2, multiple combination of hyper parameters were tested for this regression task. Using a random search instead of doing a grid search and testing all possible combination of hyper parameters allows for faster and usually better results. Indeed, it was shown that randomly selecting hyper parameters was empirically and theoritically more efficient than performing a grid search [17]. So, the metric used to compare the models of this random search was again the mean square error between the predictions of the models and the actual excess return of a company.

Results We can see in table 2, that the test MSE is first, quite similar to the validation MSE, thus showing that the model is able to generalize and apply what was learned during training to absolutely new data points. Also, the best set of hyper parameters which obtained the best metric on the validation was a model having 50 trees each having a depth of one and built using a gamma value of 5 and a subsample of 0.6. However, it is interesting to note that multiple other models achieved the same validation accuracy than the one described before.

gamma	subsample	max_depth	n_estim	valid_score	test_mse
5	0.6	1	50	-0.592	0.588
2	1.0	1	50	-0.592	-
1.5	1.0	2	10	-0.592	-
0.5	1.0	1	50	-0.592	-
5	0.6	1	10	-0.592	-
2	1.0	1	10	-0.592	-
0.5	0.6	2	10	-0.594	-
0.5	0.8	3	10	-0.595	-
2	0.8	2	50	-0.596	-
1	1.0	4	10	-0.597	-
0.5	0.6	3	50	-0.598	-
1.5	0.6	3	50	-0.599	-
1.5	0.8	4	50	-0.599	-
1	0.8	4	50	-0.6	-
2	1.0	4	50	-0.6	-

Table 2: Gradient Boosting's hyper parameter search results

Analysis The most important features of the best model were then outputted in order to determine if some features had more or less importance than others. Clearly, by looking at figure 5, feature 11 and feature 9 seemed to have the most weight when making a prediction. Also, features 4, 7, 13, 5 and 10 seemed to have absolutely zero importance, at least for this model.

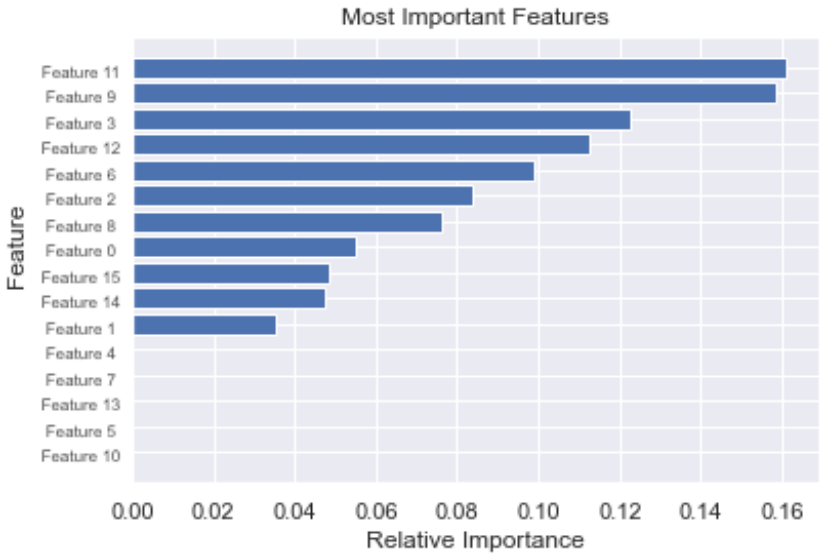


Figure 5: Gradient Boosting's most important features

Then, in order to have a clearer picture of how this gradient boosting algorithm makes decisions, three of the 50 trees present in the model were outputted and presented in figure 6. It is interesting to see the features 6, 9 and 12, three features which were shown in figure 5 as being very important, used in those trees. More particularly, this model considered that if feature 6 was smaller than -0.105 approximately, a negative value would be given to the output. Indeed, by combining the output of the 50 trees, a final value is decided for the excess return of the company.

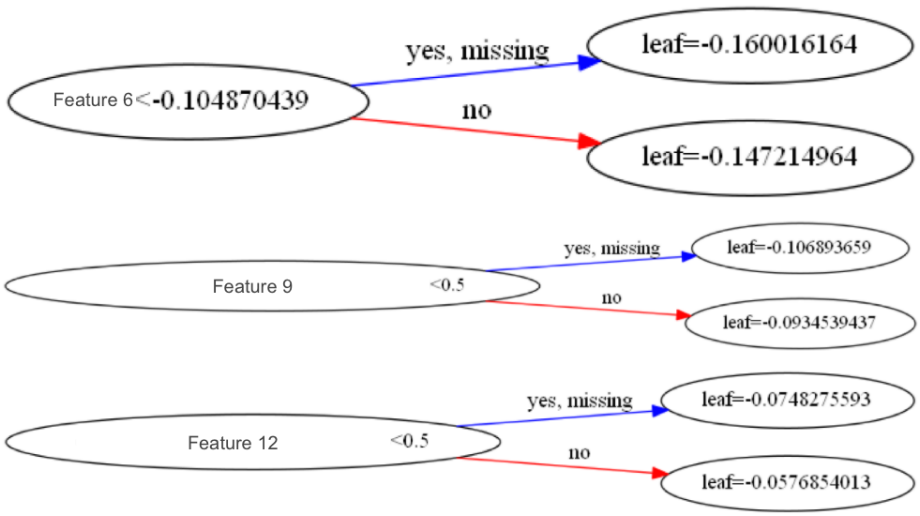


Figure 6: Gradient Boosting's Trees

The gradient boosting algorithm for the task of regression seems promising. Since this algorithm had a different test set than the ARIMAX models, their MSE cannot be compared. Indeed, the MSE of the ARIMAX models did not only include the 12 months returns, it also included, as explained previously, prediction for much closer returns, thus showing lower MSE in figure 1. However, we can see that the test MSE of the XGBoost framework outperformed by 13% the simple baseline explained in section 4.1.1 which had a MSE of 0.6649. Also, with the great functionalities of the framework which allows to show the decision trees of the model and the feature importance, the gradient boosting becomes an easier model to explain and understand. Finally, contrarily to the ARIMAX models, a unique model is used for all predictions which allows it to be versatile and easy to re-train and use in production.

4.1.4 Gradient Boosting with Time Series

Since working with time series data, it would be a loss not to try to use the information and the particularities contained in this type of data. So, with

the goal of adding the sequential aspect of the data, another gradient boosting experiment was performed.

Preprocessing In order to take advantage of the time series form of this dataset, in addition to this month's data, the k previous months' data were also used. So, in order to predict $y_{(t+12)}$, not only $y_{(t)}$ and the features $[x^{(1)} : x^{(n)}]_{(t)}$ were taken; the features $[x^{(1)} : x^{(n)}]_{([t-k]:t)}$, and $y_{([t-k]:t)}$ were also inputted to the model. For instance, if a window size of $k = 2$ was taken, the features $[x^{(1)} : x^{(n)}]_{(t-2)}$, $[x^{(1)} : x^{(n)}]_{(t-1)}$ and $[x^{(1)} : x^{(n)}]_{(t)}$ will be given as input in addition to $y_{(t-2)}$, $y_{(t-1)}$ and $y_{(t)}$ to make the prediction for $y_{(t+12)}$. In this experiment, a window size of 5 months was taken. Since the XGBoost framework did not allow for a multidimensional input, the features of the different timesteps were thus flatten and given as a one dimensional vector.

Training Methodology Similarly to the previous gradient boosting model, the dataset was separated in training, validation and test sets and a random search was performed in order to find the best set of hyper parameters, again, using the MSE as the metric.

Results By looking at table 3, we can already see a different behavior than the previous gradient boosting model. First, the best model found had different hyper parameters. Indeed, this model built with a gamma value of 1.5 and a subsample of 1.0 had 50 estimators, but a maximum depth of 3, thus having bigger trees. Also, it is quite interesting to see that in this search, not all models had the same validation MSE, as we previously saw in table 2. However, surprisingly enough, this model with a test MSE of 0.673, having a time series as input did not outperform the previous simpler gradient boosting. This observation is even more surprising considering the fact that its validation MSE was lower than the previous gradient boosting built.

gamma	subsample	max_depth	n_estim	valid_score	test_mse
1.5	1.0	3	50	-0.54	0.673
5	0.6	3	10	-0.543	-
1	0.6	3	50	-0.544	-
1.5	0.6	3	50	-0.545	-
5	1.0	3	10	-0.585	-
0.5	1.0	2	10	-0.592	-
2	1.0	2	50	-0.592	-
1.5	1.0	2	50	-0.593	-
2	0.6	2	50	-0.594	-
2	0.6	2	10	-0.594	-
1.5	0.6	2	10	-0.594	-
2	0.8	2	10	-0.597	-
5	1.0	2	50	-0.605	-
1	0.6	1	10	-0.618	-
5	1.0	1	50	-0.62	-

Table 3: Gradient Boosting with Time Series's best hyper parameters

Analysis The fact that this model had a lower validation MSE but was not able to generalize as well on the test set might be a sign of over-fitting. Indeed, it seems that by having so many features, the model had more tendency to memorize the training and validation data, instead of learning to generalize in order to apply the knowledge to new datapoints.

Another interesting feature of the XGBoost framework is the possibility to present the Shapley Additive Explanations values which assigns an importance value for each feature for a particular prediction [18]. By using the SHAP values, we can see how much and how each feature is influencing the predictions of the model. In figure 5, the importance of each feature is shown. However, just based on this graph, it is quite hard to understand how those features really influence the predictions. The use of SHAP values is not so uncommon in the financial world, especially to describe time series [19]. So, the SHAP values of this model were outputted and presented in figure 7.

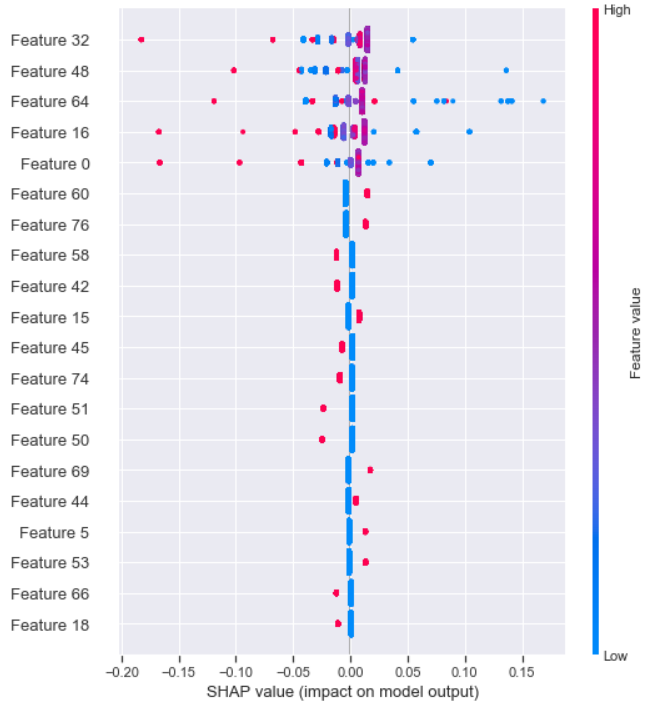


Figure 7: Gradient Boosting with Time Series SHAP values

First, we can see that some features, such as feature 60, 76, 15 and 69 clearly indicates that having a small value (shown in blue) for those features is actually hurtful for the 12 months excess return of the company (shown by a negative SHAP value). On the other hand, features 32, 48, 64, 16 and 0 seem to show mixed behaviors. Indeed, some points (which represent a single test point) for feature 32 for instance, are present in both blue and red and in both the left and right sides of the graph. This shows that there is no clear understanding if this feature having a large or small value is damaging or beneficial to the excess return of the company.

Thus, the gradient boosting algorithm used with time series data does not seem to be a great model, at least, for this specific task. Indeed, with a test MSE even worse than the simple baseline, this model seemed to have been damaged by the quantity of features added as inputs, which maybe increased the noise in the data, thus decreasing its overall performance.

4.1.5 LSTM

The LSTM, as explained in section 3.2.3 is specifically built to accept sequential data such as time series. Thus, the next built model was a LSTM built using the Keras API. One of the advantages of this architecture for time series data, compared to the XGBoost, is that it can accept a multi-dimensional input.

Indeed, there was no need to flatten the vector of input just like in the previous experiment.

Preprocessing The same data as the gradient boosting with time series was used for this experiment. Indeed, a window size of 5 months was taken with all the features considered interesting by the investment team. However, each window of features was kept in its own dimension, thus providing as input a 5-dimensional input.

Training Methodology The LSTM architecture consists of 2 hidden layers each including a 25% chance of neuron dropout, a dense layer and a sigmoid activation function at the end. This architecture was mainly based on multiple manual tests using the validation set. The models were trained using the Adam's Algorithm [20] with early stopping based on a patience value in order to reduce over-fitting when training. The hyper parameter that is the patience value determines the number of epochs of decreasing performance before stopping training. In addition to this hyper parameter, the learning rate, the number of epochs and the number of units inside each layer were also explored during this experiment using the MSE as metric and a random search.

Results The results of the hyper parameter search are shown in table 4. The best LSTM model was found using 20 epochs, a learning rate of 10^{-5}, a patience value of 3 and has 8 LSTM units. This model achieved a MSE on the test set of 0.6682, which is quite close to the validation MSE.

Epochs	Learning Rate	Lstm Units	Patience	Validation MSE	Test MSE
10	10^{-4}	4	3	0.6309	-
10	10^{-4}	4	6	0.6311	-
10	10^{-4}	8	3	0.6313	-
10	10^{-4}	8	6	0.6315	-
10	10^{-4}	16	3	0.6314	-
10	10^{-4}	16	6	0.6315	-
10	10^{-5}	4	3	0.6493	-
10	10^{-5}	4	6	0.6342	-
10	10^{-5}	8	3	0.6308	-
10	10^{-5}	8	6	0.6311	-
10	10^{-5}	16	3	0.6305	-
10	10^{-5}	16	6	0.6307	-
20	10^{-4}	4	3	0.6309	-
20	10^{-4}	4	6	0.6312	-
20	10^{-4}	8	3	0.6311	-
20	10^{-4}	8	6	0.6315	-
20	10^{-4}	16	3	0.6316	-
20	10^{-4}	16	6	0.6315	-
20	10^{-5}	4	3	0.6304	-
20	10^{-5}	4	6	0.6313	-
20	10^{-5}	8	3	0.6302	0.6682

Table 4: LSTM's hyper parameter search results

We can also see, by looking at the training curves of the best model at figure 8, that until the 8th epoch, the model's loss and MSE for both the training and validation sets were dropping quite fast showing the model's improvement. However, after the 11th epoch, it seems like the model was not able to improve, thus showing a flat curve. So, because early stopping was used for training in order to reduce the chances of over-fitting, the model's training was not allowed to continue to the 20th epoch allowed for training and stopped after 17 epochs.

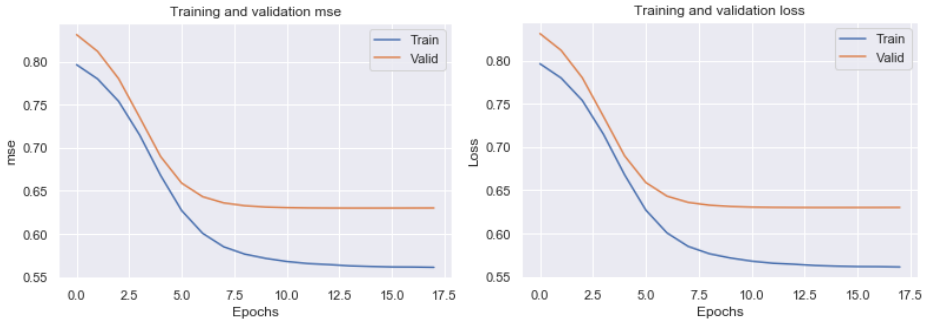


Figure 8: LSTM's training curves

Analysis As exaplained previously, contrarily to the XGBoost framework, where all features were inputted in the model as a single one dimensional vector, the LSTM's architecture allowed to input the time series as a multidimensional vector, thus separating each month's features in its own dimension. So, this allowed to output the SHAP values of the features for each time steps shown in figure 9.

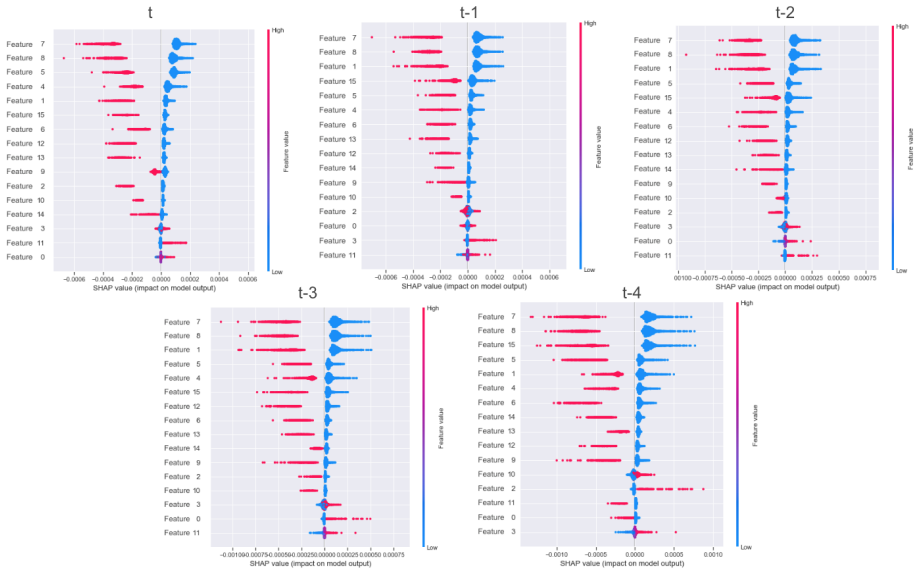


Figure 9: LSTM's SHAP values

The SHAP values presented show very clear tendencies for most of the features. For instance, having a high valued feature 2 in time step 0 is negative to the excess return. However, and what is very interesting, is that having again a high valued feature 2 at time step $t-4$ actually is positive to the excess return of the company. On the other hand, other features like feature 7, 8 and 5 stay consistent in their impact on the output, throughout the time steps.

With a MSE similar to the baseline, the LSTM model is also not the best fit for this regression task. Moreover, with a training time much higher than both the ARIMAX and the gradient boosting models, and the hardest understanbility out of the three models tested so far, this specific recurrent neural network architecture doesn't seem to be of great use for this task. However, it is important to note with neural networks in general that the architecture (e.g. the number and type of layers, the number of neurons, the activation functions) are of great impact on the performance of such models. Thus, it is hard to simply declare the LSTM has useless for this project, but much more research and testing still needs to be performed in order to find a good architecture.

4.1.6 LSTM with Clustering

The last experiment performed for the regression task was to first cluster the data and then fit a single LSTM to each cluster. To do so, the library tslearn, which is a Python package that provides machine learning tools for the analysis of time series, was used. The intuition behind this idea is that having multiple models , each specialised in a specific type of company (determined by the clustering step), would give better results than a single model fitted to the entire Universe.

Preprocessing The data used during the clustering stage is the exact same as the one used during the training of the LSTM seen in the previous experiment. So, the tslearn's input was all the features used previously, but without the targets since a look-ahead bias would be added. So, using the default values of this package, the data was separated in three clusters (0, 1 and 2). Figure 10 shows the number of datapoints for each clusters in the training, validation and test sets.

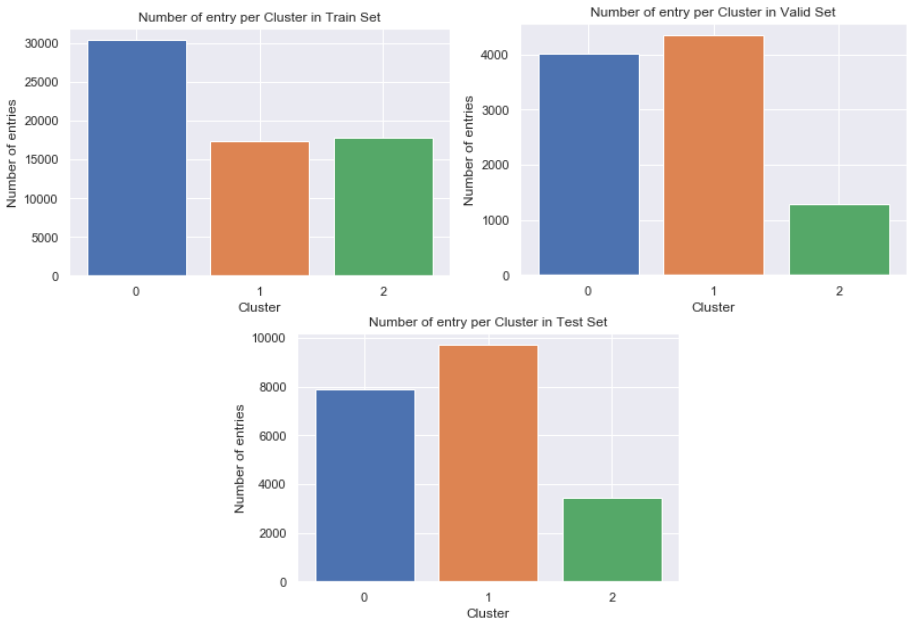


Figure 10: Cluster distribution in train, validation and test sets

We can see the distributions between the clusters are quite unequal. Indeed, cluster 0 has the most number of datapoints assigned to it in total. However, in both validation and test sets, cluster 1 has the most points. This suggests, since all the training data is temporarly older (refer to figure 1) than both the validation and test data, that cluster 0 contains mostly datapoints coming from 2009 and before. This is interesting since it suggests that the tslearn's algorithm uses temporality as a main decision criteria to form the clusters.

Training Methodology Having now each cluster separated in three splits of train, validation and test sets, a hyper parameter search was performed on each cluster in order to find the best LSTM model specific to its cluster. Again, the MSE metric was used and the same hyper parameters as the previous experiment were explored.

Results We can see in table 5 the results of the hyper parameter random search for each cluster. First, it is interesting to see that each cluster required a different set of hyper parameters. Indeed, for cluster 0, a LSTM with 4 units was created. However, the two other clusters' LSTM required 8 and 16 units for cluster 1 and 2 respectively. Also, we can see that the performance of the three models were quite different. Indeed, cluster 1's LSTM showed the worst performance with its MSE of 0.8329, compared to the two other models. By taking a weighted average of the test MSE by considering the number of points in each cluster, an average MSE of 0.6519 is achieved over the three models.

Epochs	Learning Rate	Lstm Units	Patience	Cluster 0		Cluster 1		Cluster 2	
				Validation MSE	Test MSE	Validation MSE	Test MSE	Validation MSE	Test MSE
10	10^{-4}	4	3	0.4779	-	0.7905	-	0.2903	-
10	10^{-4}	4	6	0.4784	-	0.7914	-	0.2902	-
10	10^{-4}	8	3	0.4785	-	0.7914	-	0.2902	-
10	10^{-4}	8	6	0.4789	-	0.7918	-	0.2901	-
10	10^{-4}	16	3	0.4787	-	0.7920	-	0.2902	-
10	10^{-4}	16	6	0.4787	-	0.7921	-	0.2901	-
10	10^{-5}	4	3	0.4857	-	0.7981	-	0.3825	-
10	10^{-5}	4	6	0.4824	-	0.7958	-	0.3190	-
10	10^{-5}	8	3	0.4773	-	0.7889	-	0.2945	-
10	10^{-5}	8	6	0.4770	-	0.7883	0.8329	0.2942	-
10	10^{-5}	16	3	0.4777	-	0.7911	-	0.2919	-
10	10^{-5}	16	6	0.4781	-	0.7908	-	0.2905	-
20	10^{-4}	4	3	0.4778	-	0.7904	-	0.2905	-
20	10^{-4}	4	6	0.4785	-	0.7916	-	0.2902	-
20	10^{-4}	8	3	0.4785	-	0.7915	-	0.2902	-
20	10^{-4}	8	6	0.4788	-	0.7918	-	0.2902	-
20	10^{-4}	16	3	0.4789	-	0.7921	-	0.2901	-
20	10^{-4}	16	6	0.4789	-	0.7920	-	0.2901	0.4867
20	10^{-5}	4	3	0.4771	-	0.7883	-	0.2930	-
20	10^{-5}	4	6	0.4770	0.5469	0.7887	-	0.2933	-
20	10^{-5}	8	3	0.4775	-	0.7893	-	0.2909	-
20	10^{-5}	8	6	0.4780	-	0.7903	-	0.2904	-
20	10^{-5}	16	3	0.4779	-	0.7912	-	0.2908	-
20	10^{-5}	16	6	0.4783	-	0.7883	-	0.2908	-

Table 5: LSTM's MSE for each cluster

The learning curves of the best models found in each cluster is shown in figure 11. We can see that the training evolution of cluster 0 and 1 were similar with an overall decrease in both loss and MSE. On the other hand, the LSTM trained on cluster 2 showed quite a unique evolution. Indeed, a quick drop in both training loss and MSE is shown after just one epoch and stayed stable through the end of training. Also, the validation MSE and loss seemed to stay stable throughout training.

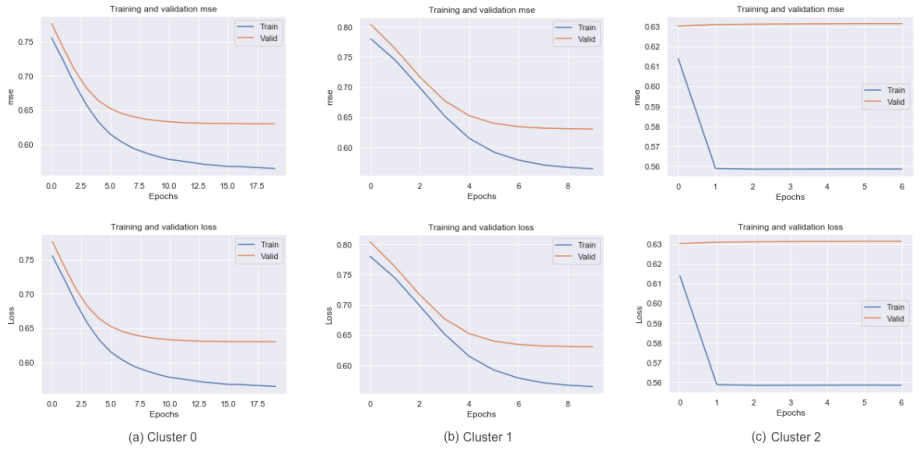


Figure 11: LSTMs' training curves for each cluster

Analysis Creating clusters with the goal of having a specialized LSTM for different time of datapoints was interesting to explore. This methodology allowed for a slight improvement of the performance compared to the singular LSTM. This can be due to the fact that the model now only had to specialize in a certain type of cluster. It was also arguable that the number of datapoints left for training was reduced for each model and thus would negatively affect the performance of the models. However, considering the quantity of datapoints available for this project and the clear improvement of the MSE seen, this effect is negligible. Finally, by increasing the number of LSTM models built, the computation time also tremendously increased since now three LSTM training and hyper parameter searches needed to be performed: a high cost for this slight increase in performance.

4.2 Classification Task

The second task considered during this project was a binary classification task. A label of zero was given to all entries with a negative 12 months excess return and a value of one for all entries with a positive 12 months excess return. Having a model able to correctly predict if a company's excess return will be positive or negative can allow an investor to choose specific companies within a Universe or even stop investing in a company. For this task, four models were tested: the same ones as the previous section except for the ARIMAX model since this model is mainly built for regression problems.

4.2.1 Baseline

The baseline taken for this task was simply a majority model. Indeed, using the training set, the class being the most popular was determined and its class simply outputted for all predictions of the test set. To get an even better idea of

the class distribution, the training and validation sets were combined in order to find the most popular class. Figure 12 shows that class 0 is actually the most popular. Thus, by always outputting a value of 0 for the predictions on the test set, an accuracy of 58.40% was achieved.

Note that even though this accuracy seems quite high (at least in the financial world) it is maybe not the most relevant model. If an investor would follow the suggestions of this baseline, it would never invest in any of the company of this dataset. However, this is not necessarily a bad methodology. Indeed, let's consider the case where all or at least the majority of the companies in this Universe are actually getting negative returns, then it is indeed a good decision to no invest in any of those companies. More specifically, if it is seen for the past years that a majority of the companies from this Universe gives negative returns, there is quite a high probability that this Universe will still continue to do so in the future.

One of the ways to really determine the performance of the models in finances, would be to perform an investment simulation: something that will be indeed explored later on in this report. So, even though this baseline seems quite naive and simple, it could still, at least, save a portfolio from bad investment decisions.



Figure 12: Class distribution in training and validation sets

4.2.2 Gradient Boosting

Being the model which obtained the best performance for the regression task, it will be interesting to observe its results on a different task. So, this algorithm was the first model tested for this new task.

Preprocessing Similarly to experiment 4.1.3, the best features suggested by the investment team were taken and the target, as explained previously, was set to be the binary value determining if the excess return is positive or negative.

Training Methodology Using the training, validation and test sets shown in figure 1, a random search was performed in order to find the best set of hyper parameters. However, contrarily to the regression problem where the mean square error was used, the metric used for this classification was the precision. The precision was preferred out of other metrics such as the F1 or even the Area Under the Receiver Operating Characteristic Curve (ROC AUC) since the main objective with this model was to find as many good performing companies as possible.

Results The results of the random search is presented in table 6. We can see here that on the validation set, the model based on 10 estimators with a maximum depths of 4, a gamma value of 2 and a subsample of 0.8, was able to achieve an accuracy of 60.30% on the validation set. We can also consider this model to generalize quite well since its accuracy on the test set was of 58.90%.

gamma	subsample	max_depth	n_estim	valid_score	test_score
2	0.8	4	10	0.603	0.589
5	0.8	4	10	0.597	-
1	0.8	1	10	0.596	-
1.5	0.8	1	10	0.596	-
1.5	0.8	3	10	0.595	-
1	1.0	1	10	0.595	-
1	0.8	3	50	0.594	-
1	0.6	2	50	0.594	-
1	0.8	3	10	0.594	-
1.5	1.0	3	50	0.594	-
2	0.8	2	10	0.593	-
1	0.6	3	10	0.593	-
5	0.8	3	10	0.593	-
0.5	0.8	2	50	0.592	-
1.5	0.8	2	10	0.592	-

Table 6: Gradient Boosting's Hyper Parameters

In order to have a clearer picture of the predictions made during the testing phase, the confusion matrix, presented in figure 13 was outputted. By looking at this matrix, we can understand that the model had a better ability to detect companies with a negative excess return. Indeed, it was able to identify almost

60% of the points labelled 0. However, it was only able to correctly identify 52.25% of the points labelled 1.

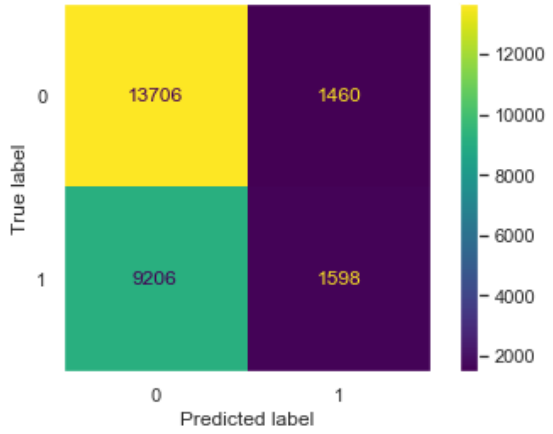


Figure 13: Gradient Boosting's Confusion Matrix

Analysis The trees of this model, being too deep and too large, couldn't be shown in this report since they would be unreadable, however the feature importance could be outputted and shown in figure 14. By comparing this model's feature importance with the gradient boosting's feature importance built for the regression task shown in figure 5, we can see some similitude.

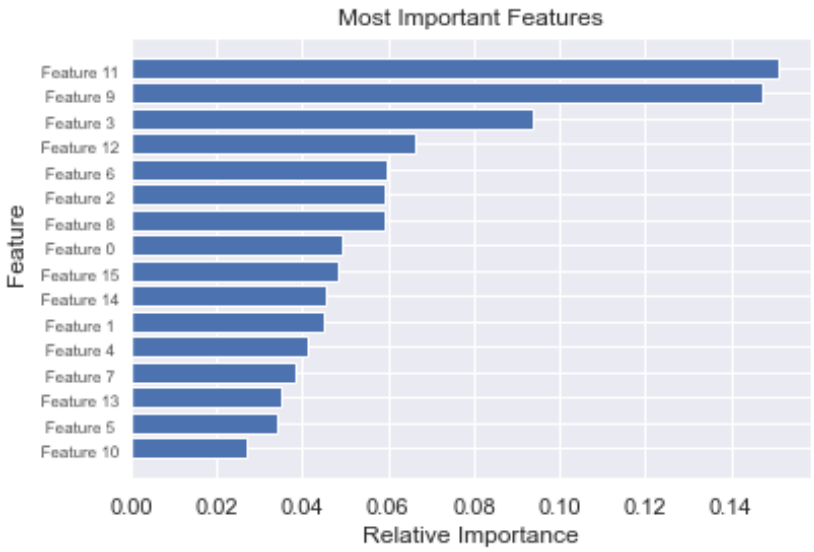


Figure 14: Gradient Boosting's Feature Importance

Indeed, once again, the features 11, 9, 3, 12 and 6 are the features which had the most importance in both tasks. Similarly, the features 10, 5, 13 and 7 were the least important. However, it is interesting to see that the features 10, 5 and 13, which were not used at all by the model during the regression task, have here a purpose during decision making. This can be due to the fact that this model had larger trees and thus learned most specific rules of classification within the trees.

Even though the test accuracy of this model was similar to the accuracy of the naive baseline of this task, by looking at the confusion matrix, it is clear that the model is able to point to some of the best companies within the dataset and is also able to determine at almost 60% accuracy if a company will have a 12 months return lower than the average of the Universe. By combining once again the fact that it is an algorithm quite fast to train, allowing an easy understandability and able to be applied to different companies, the XGBoost framework stays a very interesting alternative even for this classification task.

4.2.3 Gradient Boosting with Time Series

Just like in experiment 4.1.4, the gradient boosting was used but with time series input. Even tough during the regression task the performance of this model decreased when using this form of data, the behavior could be different for the classification task.

Preprocessing So, a window size of 5 months was taken and the features of each month were flatten in a one dimensional vector of features and given to the model.

Training Methodology A random search was performed using the accuracy as the main metric and the training methodology of experiment 4.1.4.

Results The results of the search outputted a model constructed of 10 estimators, with a maximum depth of 3, a subsample of 0.8 and a gamma value of 2. This model which achieved a validation score of 59.20% actually got better results on the test set with an accuracy of 59.40%. Again, this shows a great potential of this model's capacity to generalize to new datapoints.

gamma	subsample	max_depth	n_estim	valid_score	test_score
2	0.8	3	10	0.592	0.594
0.5	0.8	3	50	0.588	-
2	1.0	3	10	0.585	-
5	0.8	2	10	0.583	-
1	1.0	2	10	0.582	-
0.5	1.0	2	10	0.582	-
5	0.6	2	10	0.579	-
5	1.0	2	50	0.579	-
2	0.8	2	50	0.578	-
2	0.6	2	10	0.577	-
0.5	0.6	2	50	0.577	-
0.5	0.6	2	10	0.577	-
1.5	1.0	1	50	0.576	-
1	1.0	1	50	0.576	-
1	0.8	1	10	0.574	-

Table 7: Gradient Boosting with time series' hyper parameters

Analysis By looking at the SHAP values of this model presented in figure 15, we can see similar values to the regression task's gradient boosting with time series shown in figure 7. Indeed, features 48, 32, 64, 16 and 0 seem to give mixed signals in both the regression and classification tasks. Also, the effect of features 50 and 66 on the predictions of the models stayed the same in both figures. Indeed, for the two features, having a high value for this feature means a negative influence on the probability of this company showing a positive excess return. So, interestingly enough, the models seemed to have made the same conclusions for most of the features and this, even though the target and task were different.

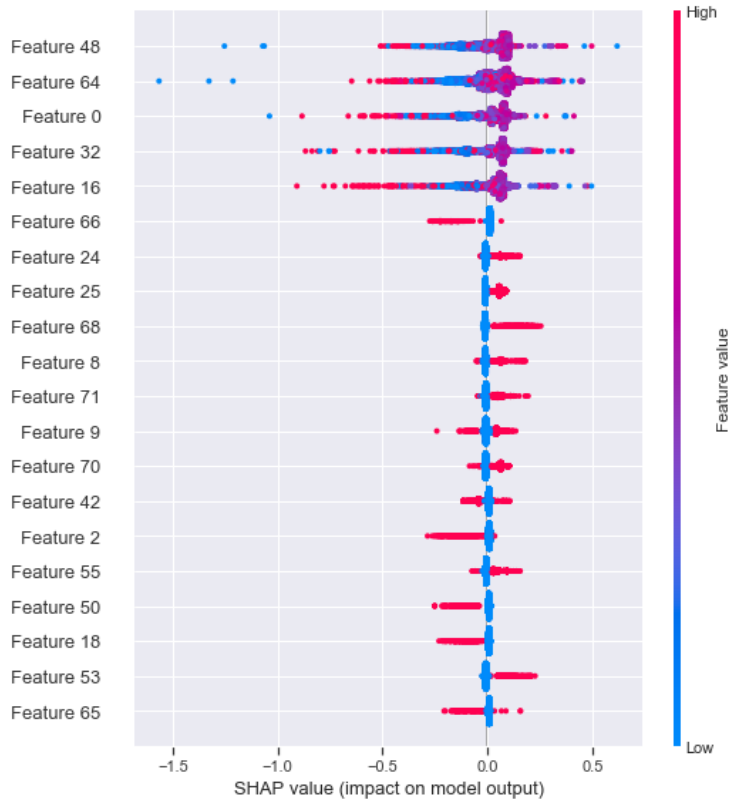


Figure 15: Gradient Boosting with time series' SHAP values

Compared to the regression task, giving the data as a time series actually increased the performance of this gradient boosting algorithm by 0.5%. However, this not very significant increase of accuracy could not justify the added complexity of having five times more features inputted to the model. Furthermore, because this increase in accuracy is so minimal, the use of a different test set could actually provide a different outcome and prove that the gradient boosting built in the previous experiment could outperform this particular one.

4.2.4 LSTM

The next model used for this classification task was the LSTM. Following the same steps and the overall architecture as the model of experiment 4.1.5, this recurrent neural network was implemented but with the target set as a binary value.

Preprocessing The preprocessing performed to obtain the features and targets were the exact same as experiment 4.1.5.

Training Methodology Using the accuracy as the metric, a random search was performed in order to find the best hyper parameters for this LSTM.

Results From table 8, we can see that the best performing LSTM was built with a patience value of 6, a learning rate of 10^{-4} and with 4 units in each layers. Its validation accuracy was 56.79% and actually lower than the test accuracy of 59.59% by more than 3%.

Epochs	Learning Rate	Lstm Units	Patience	Validation Accuracy	Test Accuracy
10	10^{-4}	4	3	0.5678	-
10	10^{-4}	4	6	0.5639	-
10	10^{-4}	8	3	0.5642	-
10	10^{-4}	8	6	0.5635	-
10	10^{-4}	16	3	0.5657	-
10	10^{-4}	16	6	0.5636	-
10	10^{-5}	4	3	0.5651	-
10	10^{-5}	4	6	0.5648	-
10	10^{-5}	8	3	0.5648	-
10	10^{-5}	8	6	0.5648	-
10	10^{-5}	16	3	0.5648	-
10	10^{-5}	16	6	0.5648	-
20	10^{-4}	4	3	0.5657	-
20	10^{-4}	4	6	0.5679	0.5959
20	10^{-4}	8	3	0.5674	-
20	10^{-4}	8	6	0.5661	-
20	10^{-4}	16	3	0.5621	-
20	10^{-4}	16	6	0.5606	-
20	10^{-5}	4	3	0.5648	-
20	10^{-5}	4	6	0.5636	-

Table 8: LSTM hyper parameter search results

We can see in figure 16 an interesting behavior by comparing the accuracy and loss evolution. In the accuracy evolution, we first see, after two epochs, that even though the training accuracy is increasing, the validation is slowly decreasing: an overall sign of over-fitting. However, on the other hand, the loss keeps on decreasing. This is a perfect example showing that a decrease in loss does not always equals better performance.

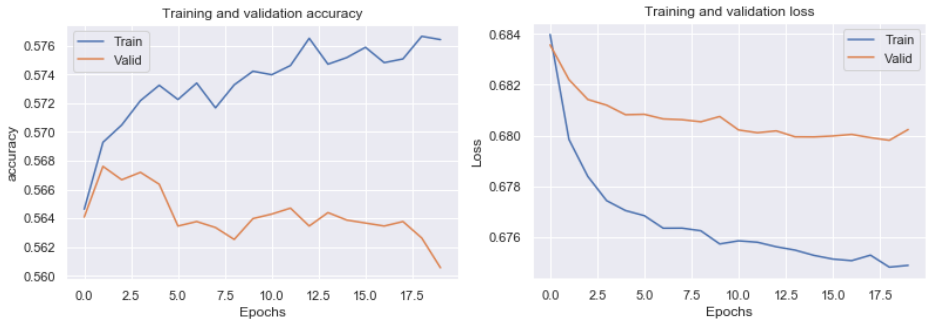


Figure 16: LSTM's training curves

Analysis Since the data was given as a five dimensional vector, again, the SHAP values for each time steps could be outputted and presented in figure 17.

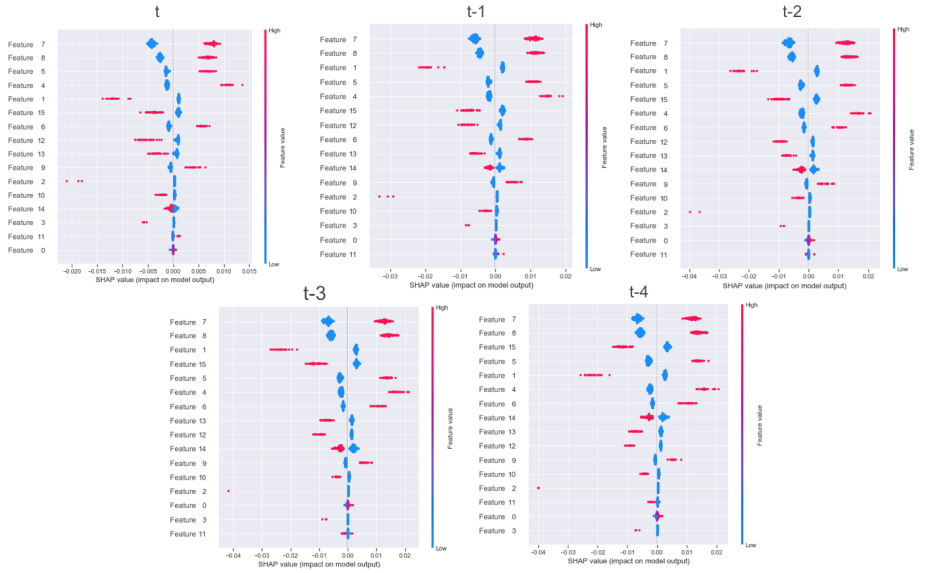


Figure 17: LSTM SHAP values

As seen before, depending on the dimension which a feature is present, its effect on the classification prediction might differ. Indeed, for instance, we can see that feature 11 in time step 0, had a positive impact if its value was high. However, this same feature in time step $t-4$ actually has a negative impact when high. Another interesting observation is that, as seen in figure 9, most of the features have a very clear impact on the classification. Indeed, except for the last 6 features on the figure, the other features are far from the centered line, thus showing for instance that a high value in feature 7, 8, 5 and 4 is always

considered positive and having a smaller value in those same features always has a negative impact on the probability that the company is part of class 1.

Thus far, the LSTM is the model having the best accuracy for this classification task, beating the gradient boosting with time series by 0.19%. However, this increase in performance does not justify the much longer time required to train this model and its complexity compared to the gradient boosting algorithm.

4.2.5 LSTM with Clustering

Finally, the last model tested for the classification task was the LSTM with clustering. Using the default inputs of the the tslearn's clustering library, three clusters were created and three different LSTM were constructed and optimized.

Preprocessing Figure 18 shows the clusters' distributions in the training, validation and test sets. First, we can see, quite similarly to the experiment 4.1.6 that one of the clusters is heavily made of points before 2009: in this case it is cluster 1 and not cluster 0 as seen previously in the regression experience (figure 10. However, the cluster labels could be changed or be interchangeable. Thus, by switching the labels of the clusters 0 and 1, the same distribution as seen in figure 10 is obtained. This behavior is expected since the clusters do not depend on the target of the task, thus overall, similar clusters should've been returned in both the regression and classification tasks: which is indeed the case.

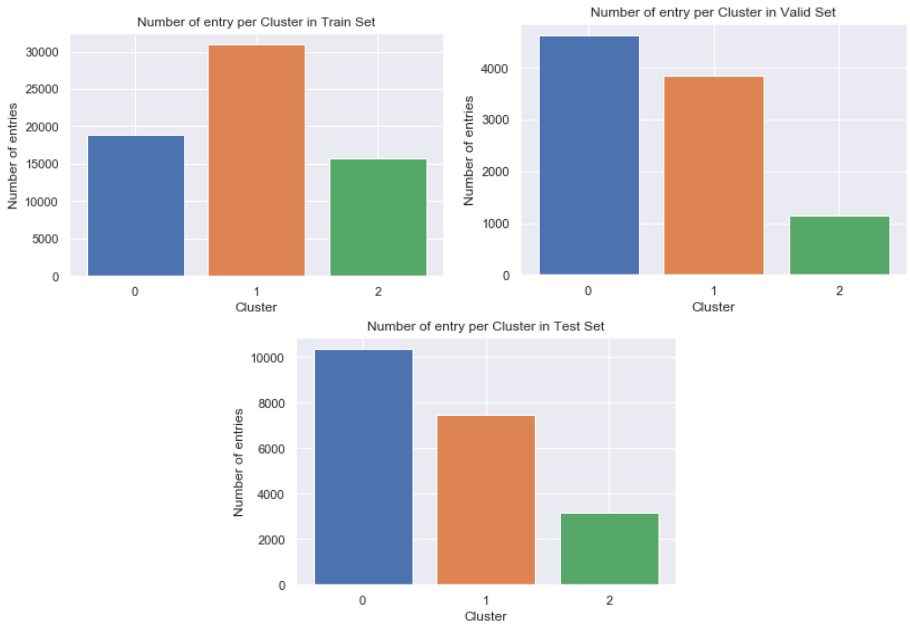


Figure 18: Cluster distribution in train, validation and test sets

Training Methodology A random search was then performed on each cluster to find the best LSTM's hyper parameters for each cluster. The best values of epochs, learning rate, number of LSTM units and patience value for the early stopping were once again the hyper parameters explored during the search.

Results We can see in table 9, the performance and best hyper parameters for each LSTM models. It is always interesting to see, how the requirements of each cluster creates a different set of best hyper parameters. The similarity between the three models is that they all use a learning rate of 10^{-4}. The model built with cluster 2 was the best performing model out of the three with a test accuracy of 53.46% and the cluster zero's model the worst with a test accuracy of 51.13%.

Epochs	Learning Rate	Lstm Units	Patience	Cluster 0		Cluster 1		Cluster 2	
				Validation Accuracy	Test Accuracy	Validation Accuracy	Test Accuracy	Validation Accuracy	Test Accuracy
10	10^{-4}	4	3	0.5226	-	0.5375	-	0.5264	-
10	10^{-4}	4	6	0.5389	0.5113	0.5235	-	0.5385	-
10	10^{-4}	8	3	0.5243	-	0.5277	-	0.5333	-
10	10^{-4}	8	6	0.5275	-	0.5375	-	0.5359	-
10	10^{-4}	16	3	0.5273	-	0.5279	-	0.5316	-
10	10^{-4}	16	6	0.5266	-	0.5305	-	0.5255	-
10	10^{-5}	4	3	0.5273	-	0.5124	-	0.5108	-
10	10^{-5}	4	6	0.5255	-	0.5124	-	0.5108	-
10	10^{-5}	8	3	0.5273	-	0.5121	-	0.5108	-
10	10^{-5}	8	6	0.5273	-	0.5124		0.5108	-
10	10^{-5}	16	3	0.5273	-	0.5124	-	0.5108	-
10	10^{-5}	16	6	0.5273	-	0.5119	-	0.5108	-
20	10^{-4}	4	3	0.5318	-	0.5331	-	0.5411	0.5346
20	10^{-4}	4	6	0.5307	-	0.5295	-	0.5359	-
20	10^{-4}	8	3	0.5338	-	0.5318	-	0.5376	-
20	10^{-4}	8	6	0.5290	-	0.5386	0.5472	0.5367	-
20	10^{-4}	16	3	0.5333	-	0.5360	-	0.5350	-
20	10^{-4}	16	6	0.5329	-	0.5124	-	0.5393	-
20	10^{-5}	4	3	0.5273	-	0.5129	-	0.5108	-
20	10^{-5}	4	6	0.5273	-	0.5134	-	0.5108	-
20	10^{-5}	8	3	0.5273	-	0.5124	-	0.5108	-
20	10^{-5}	8	6	0.5273	-	0.5139	-	0.5108	-

Table 9: LSTMs' Accuracy for each cluster

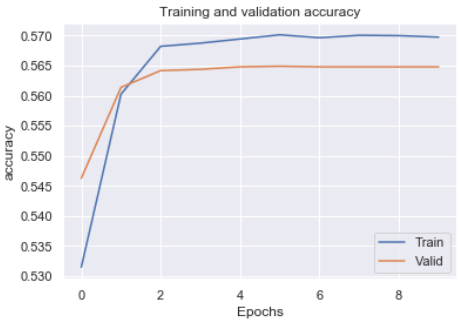
The training curves of the models, shown in figure 19 are also quite interesting to compare. Indeed, the models built on cluster 1 and 2 showed a different accuracy evolution than the LSTM built on cluster 0. However, the three models' loss evolution were similar: even at the end of training, both the training and validation losses kept on decreasing, but since the early stopping condition is based on the accuracy of the models, training was stopped since the models didn't seem to have an increase in their performance.



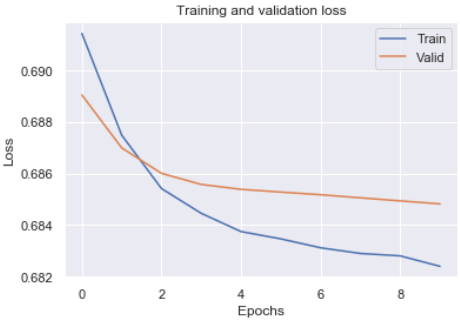
(a) LSTM's Accuracy on Cluster 0



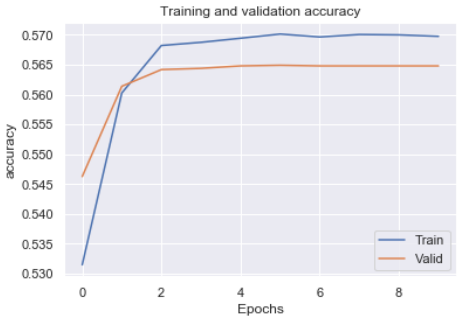
(b) LSTM's Loss on Cluster 0



(c) LSTM's Accuracy on Cluster 1



(d) LSTM's Loss on Cluster 1



(e) LSTM's Accuracy on Cluster 2



(f) LSTM's Loss on Cluster 2

Figure 19: LSTM's training curves for each Cluster

Analysis In addition of having the worst performance out of all the models tested in this classification task, the LSTM based on clustering was also the most expensive in time and resources. Indeed, with accuracies which did not even pass 55%, this model didn't seem to perform as well as with the regression task. Thus, those models were not considered interesting for this task.

4.3 Ranking Task

The last task implemented for this project was a ranking task. The goal would be, for each month, to rank all the companies present within this month based on their potential 12 months excess return. Thus, the company ranked first is considered to have the highest probability of having the highest 12 months excess return. This system would filter and extract the best companies of the month and thus allow an investor to further explore them and make a final decision. A ranker model is a good middle ground between a fully automated investment system and a totally discretionary investment process.

4.3.1 Gradient Boosting

Built toward the end of the internship, only a single ranker, using the gradient boosting algorithm, was used. Indeed, the framework XGBoost has a module already implementing a ranker algorithm. The understanbility of this model was the main driver behind using it instead of other models such as the LSTM or other machine learning models.

Also, since the ranker model seemed to be the model which most interested the investors, an investment simulation was performed by following for multiple years the suggestions of the model in order to see the potential returns made by the model.

Preprocessing Since the data was processed to be of a monthly frequency, the raking of the companies needed to be determined once every months. So, a label had to be added to every points of the dataset. All the datapoints which were part of the same month were labelled with the same flag. This way, the XGBoost framework could process which points to consider together. So, the features, the tags and the 12 months exess return for each company, each month, were the inputs of this model.

Training Methodology The training, validation and test sets contained the same years as what is presented in figure 1. Even though the XGBoost's module used is different from the classifier and the regression modules, it still uses the gradient boosting algorithm and thus the same hyper parameters are used in this ranker. The training implementation used by default by this module is to minimize the pairwise loss. Thus, the goal for the ranker is to minimize the number of inversions in ranking: cases where the pair of results are in the wrong order relative to the ground truth.

However, since the main goal was to implement the simulation described in the next section, not much time was dedicated to the search of hyper parameters and only a model using the default values of XGBoost was implemented as time of writing.

Results The first simulation performed is shown in figure 20. In this simulation, for each month of the test set (i.e. never seen before data), a ranking of the

companies was computed. Then, at each month, an investment would be made on the top 20 companies returned by the model.



Figure 20: Investment Simulation

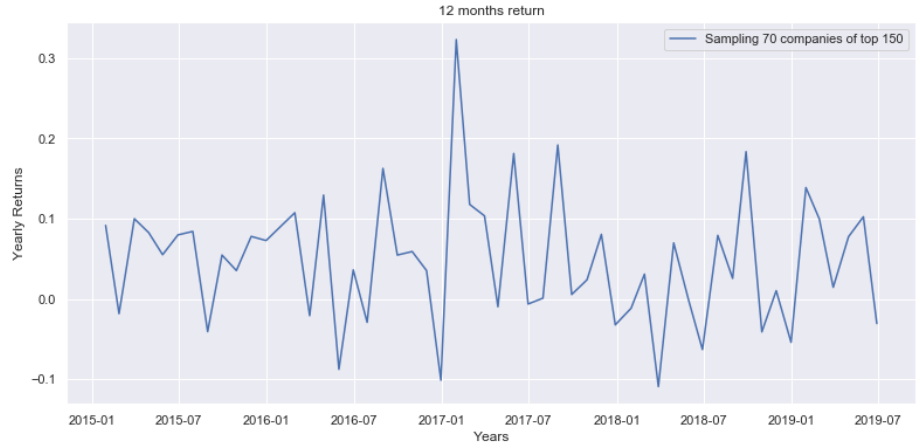
This process is repeated every month throughout the duration of the test set. Then, the same methodology would be applied but for the top 50, 100 and

150 companies. It is important to note that the return percentages seen here are excess returns. In other words, this compare the returned gained to the investment in the entire Universe provided.

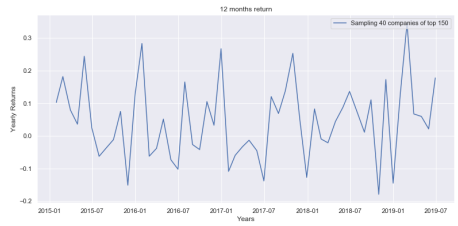
As seen in figure 20, the best results were achieved by following the top 20 companies. Indeed, on average, a 12 months excess return of 8.17% is gained when following the top 20 companies, compared to the 6.496%, 6.553% and 5.61% for the top 50, 100 and 150 respectively. Moreover, by looking at the graph of the top 20 companies, we can see that if in May 2017, one would've followed the model's suggestions and invested in the top 20 companies, an excess return of more than 60% would've been achieved one year after (in May 2017). However, around June 2019, if one would've invested in the top 20 companies suggested, an excess return of almost -40% would've been seen one year later. It is however important to see that overall, a positive excess return is achieved throughout the years.

In order to reduce this difference from one year to another and have less volatility in the results, one could decide to diversify the investment made by following more companies. So, even though the average excess return is lower when investing in the top 150 companies, less draw downs are seen. Indeed, the loss seen in June 2019 with only 20 companies could've been reduced by more than 45% (thus having a positive excess return of 5%) if the investment was made instead in the top 150 companies. However, this would be at the cost of a 1% reduction on the average excess return.

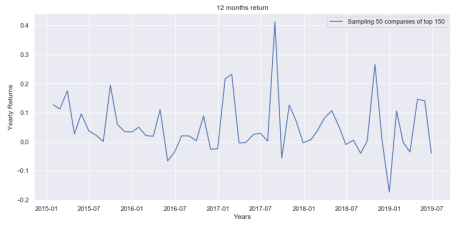
In the second simulation, instead of directly investing in the top 150 companies, a specific number of companies will be randomly sampled each month and invested into. First, 20 companies will be sampled out of the top 150, then 40, 50, 60, 70 and then 100 companies. By randomly sampling companies out of the top 150, this would suggest that the model was indeed capable of highlighting the most interesting companies and that the results of positive returns is not mainly due to the high number of companies invested into or to some luck or randomness.



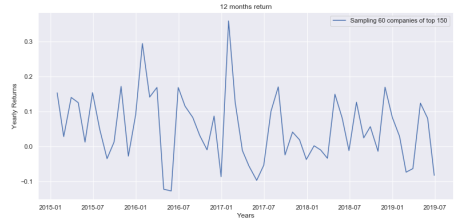
On average: 0.0485155725998252



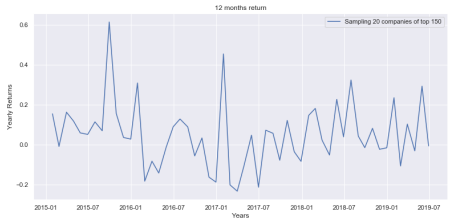
On average: 0.04512010370230267



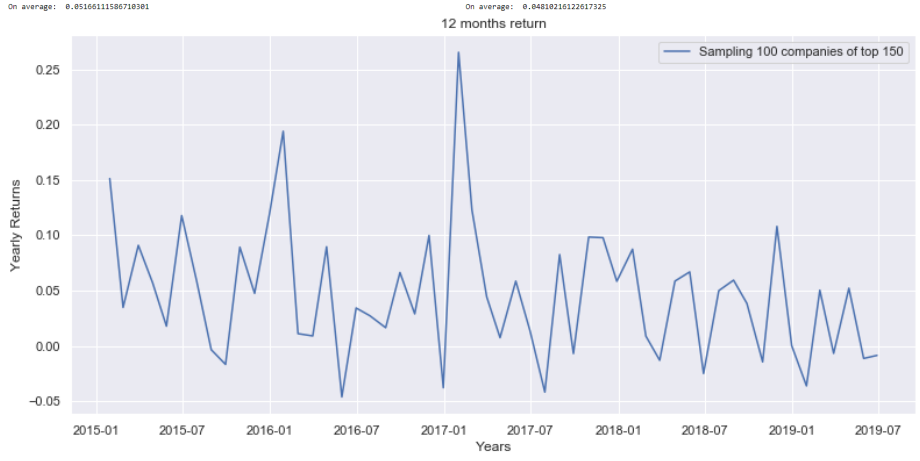
On average: 0.05194725354236584



On average: 0.05166111586710301



On average: 0.04810216122617325



On average: 0.04673601768255509

Figure 21: Sampling from the top 150 companies

We can see some interesting results in figure 21. First, in all of those simulations, the average 12 month excess returns are positive, showing that it is overall more advantageous to follow the suggestions of the model, instead of investing in the entire Universe. By looking for instance at the case where 70 companies were sampled, an average excess return of 4.851% was achieved, which isn't too far from the results obtained in figure 20 when 150 companies were taken. Similar results were obtained for the other amounts of companies sampled.

Analysis Clearly, based on those results, the gradient boosting as a recommendation system for investors seems like a very interesting and profitable system. Indeed, the results presented show that by following the suggestions of the model, a better return can be achieved than to invest in all the companies present within that Universe. However, those results are based on some assumptions. Indeed, no transaction cost or fees were considered. Also, when the top companies were selected, all of them were equally invested into, which is not always possible due for instance to the quantity of stocks available or simply due to the investor's decisions.

Nonetheless, the ranking system seemed like the best solution out of the three tasks explored during this project (the regression, classification and ranking tasks). Indeed, such systems allow to combine the potential of artificial intelligence with the experience, expertise and discretion of a professional investor, things which are still (for now) hard to capture in any machine learning models alone.

5 Recommendations

This project still being an ongoing initiative has many opportunities to improve. First, many other models could have the potential of getting good performance in each of the tasks explored. For instance, it would've been interesting to explore a neural network for the task of ranking, since only a single algorithm was tested. Indeed, a system could be built using a LSTM with the triplet loss which takes three points (a positive point, a center and a negative point) and pulls the center and positive points together while pushing the center and negative points away from each other [21].

Another improvement that could be made is to perform a simulation on the two other tasks tackled during this project, namely the regression and classification tasks. In the case of the regression models, we can every month invest in all the companies which the models predicted to have a positive 12 months return. Similarly, in the classification task, we could simulate investing in all the companies which were predicted to be of class 1 (thus having a positive excess return). Those results could then be compared with the results seen from the ranking task.

Finally, another feature that could also be explored would be the combination of the models coming from different tasks. For instance, when considering the top companies returned by the ranker, an added step of filtering could be

performed by only selecting the companies which were also labeled as a class 1 by the classifier. It would be interesting to see if the classifier considered all the companies part of ranker's top suggestions as potentially getting a positive excess return for instance. Furthermore, the regression model could be added to set a specific threshold for the expected return of the company. For instance, even if a company is labeled as a 1 and part of the ranker's suggestions, this company would only be considered if its predicted return is above 5%.

6 Conclusion

This internship allowed me to first apply the theoretical and practical knowledge gained during the different classes and projects completed during my academic career. More specifically, doing an internship at CDPQ, especially during a financial crisis, was exceptionally rewarding. In addition to learning multiple new tools, the coding and research methodology used within the company and the very interesting field that is machine learning in financial institutions, one of the great lesson learned during this project was the importance of vulgarisation and explanation of the results.

Indeed, especially when the use of machine learning is of great importance, such as in financial institutions, in the healthcare system or even in the education system, it is very important to clearly understand the model used and how its decisions are made. In such fields, it would be irresponsible to blindly follow the predictions of a model, even though the results on a test set are spectacular. So, I learned that even though the results of a particular system can be spectacular, if its understanbility and explicability is limited (e.g. for people having a limited background in artificial intelligence) a simpler model with adequate results will be preferred.

Finally, to conclude, closely working with a team with a strong knowledge in machine learning in a project which was of particular interest for me will surely be of a great advantage for my career in machine learning.

References

[1] Marcos Lopez De Prado. *Advances in financial machine learning*. John Wiley & Sons, 2018.

[2] Thomas G Dietterich. Machine learning for sequential data: A review. In *Joint IAPR international workshops on statistical techniques in pattern recognition (SPR) and structural and syntactic pattern recognition (SSPR)*, pages 15–30. Springer, 2002.

[3] F Bell and S Smyl. Forecasting at uber: An introduction. *Uber Engineering*, 2018.

[4] T. Kimoto, K. Asakawa, M. Yoda, and M. Takeoka. Stock market prediction system with modular neural networks. In *1990 IJCNN International Joint Conference on Neural Networks*, pages 1–6 vol.1, 1990. doi: 10.1109/IJCNN.1990.137535.

[5] Jianfeng Si, Arjun Mukherjee, Bing Liu, Qing Li, Huayi Li, and Xiaotie Deng. Exploiting topic based twitter sentiment for stock prediction. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 24–29, 2013.

[6] D. M. Q. Nelson, A. C. M. Pereira, and R. A. de Oliveira. Stock market's price movement prediction with lstm neural networks. In *2017 International Joint Conference on Neural Networks (IJCNN)*, pages 1419–1426, 2017. doi: 10.1109/IJCNN.2017.7966019.

[7] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo. Stock price prediction using the arima model. In *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, pages 106–112, 2014. doi: 10.1109/UKSim.2014.67.

[8] Dale W Jorgenson. Rational distributed lag functions. *Econometrica: Journal of the Econometric Society*, pages 135–149, 1966.

[9] Herman Wold. *A study in the analysis of stationary time series*. PhD thesis, Almqvist & Wiksell, 1938.

[10] Jerome H Friedman. Stochastic gradient boosting. *Computational statistics & data analysis*, 38(4):367–378, 2002.

[11] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

[12] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[13] James A Anderson. *An introduction to neural networks*. MIT press, 1995.

[14] T Mikolov, M Karafiát, L Burget, J Cernocky, and S Khudanpur. Recurrent neural network based language model, in 'eleventh annual conference of the international speech communication association'. 2010.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Lstm can solve hard long time lag problems. In *Advances in neural information processing systems*, pages 473–479, 1997.

[16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[17] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *The Journal of Machine Learning Research*, 13(1):281–305, 2012.

[18] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in neural information processing systems*, pages 4765–4774, 2017.

[19] Karim El Mokhtari, Ben Peachey Higdon, and Ayşe Başar. Interpreting financial time series with shap values. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*, pages 166–172, 2019.

[20] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[21] Jiang Wang, Yang Song, Thomas Leung, Chuck Rosenberg, Jingbin Wang, James Philbin, Bo Chen, and Ying Wu. Learning fine-grained image similarity with deep ranking. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1386–1393, 2014.